Governors: Courage — Answers

Courage

The abstract data generating mechanism coming out of Justice is:

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_n X_n + \epsilon, \quad \epsilon \sim N(0, \sigma^2)$$

Candidate Models

Estimate a model in which `lived_after` is the outcome and `election_result` is the only covariate. Display the coefficients and their confidence intervals.

```
linear_reg() |>
  fit(lived_after ~ election_result, data = x) |>
  tidy(conf.int = TRUE)
```

```
# A tibble: 2 x 7
  term               estimate std.error statistic  p.value conf.low conf.high
  <chr>                 <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)           27.2       1.26     21.5  4.20e-59    24.7      29.7
2 election_resultWin     2.39      1.72      1.39 1.67e- 1    -1.01      5.79
```

The raw win effect is about 2.4 years, but its confidence interval includes zero — comparing group averages alone, we cannot distinguish the effect from nothing. Age is the elephant in the room from the EDA, so we adjust for it next.

Estimate a model in which `lived_after` is the outcome and `election_result` and `election_age` are the covariates. Display the coefficients and their confidence intervals.

```
linear_reg() |>
  fit(lived_after ~ election_result + election_age, data = x) |>
  tidy(conf.int = TRUE)
```

```
# A tibble: 3 x 7
  term               estimate std.error statistic  p.value conf.low conf.high
  <chr>                 <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)          74.2      4.72       15.7  3.24e-39   64.9      83.5
2 election_resultWin    2.06     1.45        1.42 1.58e- 1   -0.803     4.92
3 election_age         -0.915    0.0896    -10.2  1.02e-20   -1.09     -0.739
```

Age at election is enormously informative (about −0.9 years of remaining life per year of age, interval far from zero), but the win coefficient barely moves and still cannot be distinguished from zero — consistent with the balance check in Wisdom, where winners and losers had nearly identical ages. The missing ingredient is `win_margin`: even inside the close window, winners sit above zero and losers below, so the running variable is entangled with the treatment. We control for it, and add `party`, next.

Add `win_margin` and `party` to the previous model. Display the coefficients and their confidence intervals.

```
linear_reg() |>
  fit(lived_after ~ election_result + win_margin + election_age + party, data = x) |>
  tidy(conf.int = TRUE)
```

```
# A tibble: 6 x 7
  term               estimate std.error statistic  p.value conf.low conf.high
  <chr>                 <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)          67.6      4.94       13.7  4.18e-32    57.8     77.3
2 election_resultWin    8.63     2.74        3.15 1.83e- 3     3.24    14.0
3 win_margin           -1.46     0.505      -2.89 4.21e- 3    -2.46    -0.464
4 election_age         -0.887    0.0875    -10.1  1.98e-20    -1.06    -0.715
5 partyRepublican       4.04     1.43        2.83 5.05e- 3     1.23     6.85
6 partyThird party     -9.50     8.03       -1.18 2.38e- 1   -25.3      6.31
```

Controlling for the running variable changes everything: the win coefficient jumps to about 8.6 years with an interval excluding zero. Comparing candidates *at the same margin*, on opposite sides of the win/lose threshold, is the regression-discontinuity comparison the design intends — and it is only visible once `win_margin` is in the model.

The Data Generating Mechanism

Which model should we choose, and why? Define `fit_governors` as your chosen model and write out the fitted equation.

```r
fit_governors <- linear_reg() |>
  fit(lived_after ~ election_result + win_margin + election_age + party, data = x)

fit_governors |>
  tidy(conf.int = TRUE)
```

```
# A tibble: 6 x 7
  term               estimate std.error statistic  p.value conf.low conf.high
  <chr>                 <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)          67.6      4.94       13.7  4.18e-32    57.8     77.3
2 election_resultWin    8.63     2.74        3.15 1.83e- 3     3.24    14.0
3 win_margin           -1.46     0.505      -2.89 4.21e- 3    -2.46    -0.464
4 election_age         -0.887    0.0875    -10.1  1.98e-20    -1.06    -0.715
5 partyRepublican       4.04     1.43        2.83 5.05e- 3     1.23     6.85
6 partyThird party     -9.50     8.03       -1.18 2.38e- 1   -25.3      6.31
```

We choose the full model. `win_margin` must stay because it is the running variable that identifies the design; `election_age` because it dominates the outcome; and `party` because `partyRepublican` is distinguishable from zero (Republicans in this sample live about 4 years longer after election). The interval for `partyThird party` easily includes zero, but with a categorical variable we keep the whole variable as long as at least one of its dummies earns its place — dropping only the Third-party level is not an option.

$$\widehat{\text{lived\_after}} = 67.6 + 8.63 \cdot \text{Win} - 1.46 \cdot \text{win\_margin} - 0.89 \cdot \text{election\_age} + 4.04 \cdot \text{Republican} - 9.50 \cdot \text{Third}$$

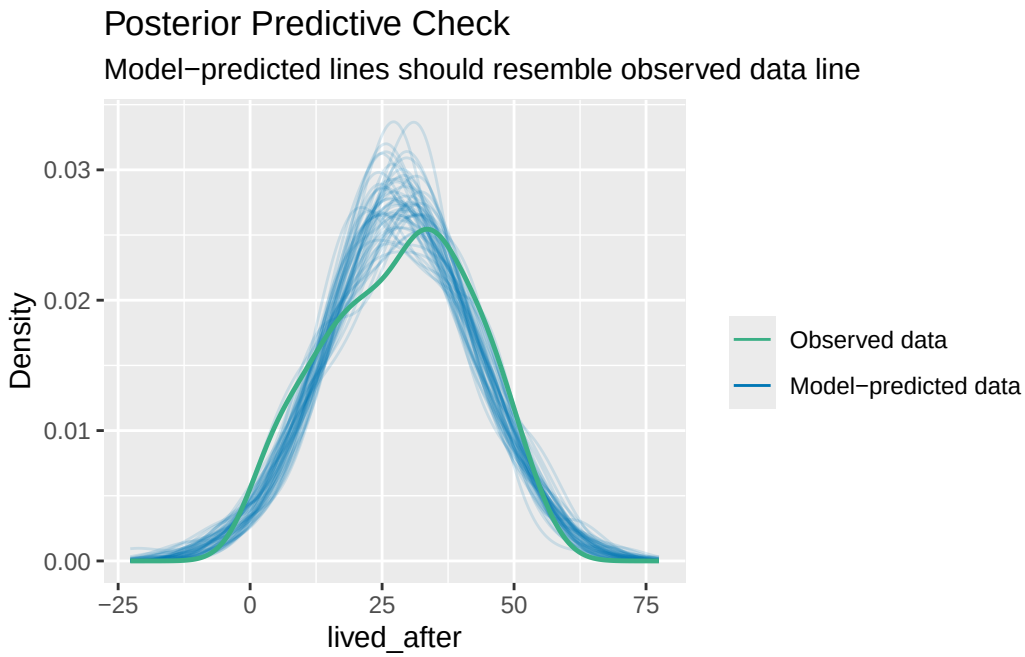
This is our data generating mechanism.

Model Checking

Use `check_predictions(extract_fit_engine(fit_governors))` to perform a posterior predictive check of your chosen model.

```r
check_predictions(extract_fit_engine(fit_governors))
```

```
Ignoring unknown labels:
* size : ""
```



`check_predictions()` simulates new lifespans from the fitted model and overlays them on the observed distribution. The simulated curves track the observed density reasonably well, so the model is not obviously misdescribing the outcome — the check we want to pass before trusting the predictions in Temperance.
